

Лаборатории Nviso

Исследования в области кибербезопасности прямо из лаборатории! 🐾



≡ Меню

Перехват трафика Flutter на Android (ARMv8)

Йерун Бекерс 📁 Мобильная безопасность , Android ⌚ 20 мая 2020 г. ≡ 7 минут

В предыдущем сообщении в блоге я объяснил свои шаги по реверсированию двоичного файла flutter.so для определения правильного смещения/шаблона для обхода проверки сертификата. В качестве очень быстрого резюме: **Flutter не использует системные настройки прокси-сервера и не использует хранилище сертификатов системы**, поэтому обычные подходы не работают. Мое предыдущее руководство объясняло только, как перехватывать Flutter на устройствах Android ARMv7, но шаги не полностью переносятся на ARMv8, поэтому в этом сообщении в блоге кратко объясняются шаги для ARMv8



Обновление август 2022 г.



Обновление к этому посту в блоге было написано и его можно найти [здесь](#) . Оно охватывает как iOS, так и Android, а также удобный скрипт/код-шеринг Frida для использования.

Этот пост в блоге написан как руководство/мыслительный процесс, поэтому краткий обзор вы найдете внизу .

Тестирование приложений

Во-первых, нам понадобится тестовое приложение. Я немного обновил предыдущее, добавив две кнопки: одну для HTTP-вызовов и одну для HTTPS-вызовов. Таким образом, я могу проверить, работает ли прокси, а затем — работает ли скрипт Frida.

Приложение [можно загрузить](#) с нашего GitHub.

В приложении есть две функции, которые вызывают конечную точку HTTP и HTTPS:

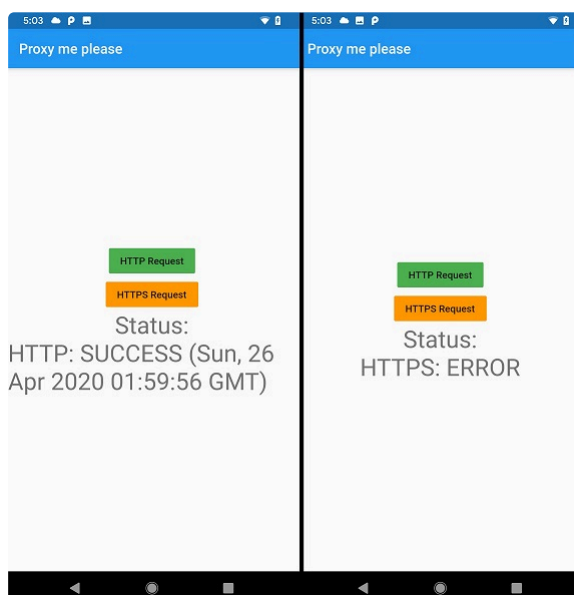
```
1 void callHTTP(){
2   client = HttpClient();
3   _status = "Calling...";
4   client
5     .getUrl(Uri.parse('http://neverssl.com'))
6     .then((request) => request.close())
7     .then((response) => setState((){_status = "HTTP:
8     .catchError((e) =>
9       setState(() {
10        _status = "HTTP: ERROR";
11        print(e.toString());
12      })
13    });
14 }
15 void callHTTPS(){
16   client = HttpClient();
17   _status = "Calling...";
18   client
19     .getUrl(Uri.parse('https://www.nviso.eu')) // pro
20     .then((request) => request.close()) // sends the
21     .then((response) => setState((){
22       _status = "HTTP
23     })))
24     .catchError((e) =>
25       setState(() {
26        _status = "HTTPS: ERROR";
27        print(e.toString());
28      })
```

```
29 |
30 | }
```

```
);
```

Проксирование приложения

Приложения Flutter по-прежнему не используют автоматически прокси-сервер системы, если только разработчик не добавит эту функциональность, создав пользовательские плагины Android и iOS, которые предоставляют эту информацию. Очевидно, что многие разработчики этого не сделают, поэтому нам по-прежнему нужно перехватывать трафик с помощью метода ProxyDroid на основе root, а не настраивать прокси-сервер WIFI через ОС Android. После настройки ProxyDroid с правильными параметрами Burp может видеть запросы от приложения.



HTTP-запросы работают без каких-либо специальных требований, тогда как HTTPS-вызов выводит ошибку в logcat:

```
04-26 16:59:02.758 11773 11802 E flutter : [ERROR:flutter/li
04-26 16:59:02.758 11773 11802 E flutter : NO_START_LIN
04-26 16:59:02.758 11773 11802 E flutter : PEM routines
04-26 16:59:02.758 11773 11802 E flutter : NO_START_LIN
04-26 16:59:02.758 11773 11802 E flutter : PEM routines
04-26 16:59:02.758 11773 11802 E flutter : CERTIFICATE_
```

Отключение проверки SSL

Сначала я думал, что версия x64 будет идентична версии x86. Это тот же исходный код, так почему же шаги должны отличаться... К сожалению, при поиске 'x509.cc' в flutter.so я нашел то же количество результатов, но ни один из них не был правильной функцией:

```

s_../third_party/boringssl/src/_001c885a      XREF[4]:      FUN_005f4e1c:005f65f4(*),
                                                FUN_0065a224:0065a3a0(*),
                                                FUN_0065a224:0065a3b4(*),
                                                FUN_0065a224:0065a400(*)
001c885a 2e 2e 2f      ds      "../third_party/boringssl/src/ssl/ssl_x509....
      2e 2e 2f
      74 68 69 ...

```

Предыдущий подход кажется неэффективным

It's pretty obvious that the [ssl_x509.cc](#) class has been compiled somewhere in the 0x650000 region, but that's still a lot of functions to try to find the correct one. If searching for the filename doesn't work, maybe searching for the line number would work. If we take a look at the [ssl_crypto_x509_session_verify_cert_chain](#) function again, we can see that the OPENSSL_PUT_ERROR macro is called at line 390. Searching for the number 390 (or 0x186) gives us some results (Search > For Scalars...):

Location	Preview	Hex	Decimal (Si...	Function Name
00103440	ddw 186h (dword[437][324])	186	390	
001aa6ea	ushort 186h	186	390	
001ab348	ushort 186h	186	390	
0037d124	add x8,x8,#0x186	186	390	FUN_0037c188
003b90a8	mov w3,#0x186	186	390	FUN_003b8f6c
0053609c	add x1,x1,#0x186	186	390	FUN_00533184
0065a604	mov w3,#0x186	186	390	FUN_0065a4ec
006e1574	add x8,x8,#0x186	186	390	FUN_006e1384
006fd854	add x9,x8,#0x186	186	390	FUN_006fd5bc
0075a5b8	cmp w0,#0x186	186	390	FUN_0075a5b8

Filter:

Min: -2147483647 Dec Max: 2147483647 Dec

Searching for magic numbers

A few of the results are around the 0x650000 region. The highlighted function (FUN_0065a4ec) looks like a good candidate, as the constant is loaded in w3 (the lower 32bit part of the x3 register), which is one of the argument registers on ARMv8. The function FUN_0065a4ec also has the correct signature, and it generally looks the same as the ARMv7 version:

<pre> C:\Decompile: FUN_003b7578 - (libflutter_32.so) 1 2 undefined4 FUN_003b7578(int iParm1,int *piParm2,undefined *puParm3) 3 4 { 5 undefined uVar1; 6 int iVar2; 7 char *pcVar3; 8 int iVar4; 9 undefined4 uVar5; 10 uint uVar6; 11 int *piVar7; 12 int iVar8; 13 undefined auStack168 [16]; 14 int iStack152; 15 int iStack140; 16 int iStack72; 17 undefined auStack44 [8]; 18 19 *puParm3 = 0x50; 20 piVar7 = *(int **) (iParm1 + 0x98); 21 if ((piVar7 == (int *)0x0) (*piVar7 == 0)) { 22 return 0; 23 } 24 iVar2 = *(int *) (*piParm2 + 0x34); 25 iVar8 = *(int *) (*piParm2[1] + 0x10) + 0x2c; 26 iVar4 = *(int *) (iVar2 + 0x4c); 27 uVar5 = *(undefined4 *)piVar7[1]; 28 __aeabi_memclr8(auStack168,0x50); 29 if (iVar8 != 0) { 30 iVar4 = iVar8; 31 } 32 iVar4 = FUN_0039d378(auStack168,iVar4,uVar5,piVar7); 33 if ((iVar4 == 0) (iVar4 = FUN_00381108(auStack44,0,*piParm2, iVar4 == 0)) { 34LAB_003b762e: 35 FUN_0037c21c(0x10,0xb,".../third_party/boringssl/src/ssl/ssl_x509.cc",0x186); 36 } 37 else { 38 pcVar3 = "ssl_server"; 39 if ((* (byte *) (*piParm2 + 0x58) & 1) != 0) { 40 pcVar3 = "ssl_client"; 41 } 42 } </pre>	x86	<pre> C:\Decompile: FUN_0065a4ec - (libflutterarm64.so) 1 2 ulonglong FUN_0065a4ec(ulonglong iParm1,undefined8 uParm2,undefined *puParm3) 3 4 { 5 int iVar1; 6 undefined8 uVar2; 7 undefined uVar3; 8 code *pcVar4; 9 ulonglong extraout_x9; 10 uint uVar5; 11 ulonglong unaff_x20; 12 undefined8 *unaff_x21; 13 ulonglong uVar6; 14 ulonglong iVar7; 15 undefined auStack312 [32]; 16 ulonglong local_118; 17 ulonglong local_100; 18 int local_88; 19 undefined auStack88 [8]; 20 21 *puParm3 = 0x50; 22 if ((* (longlong **) (iParm1 + 0xa8) == (longlong *)0x0) (** (longlong **) (iParm1 + 0xa8) == 0)) { 23 uVar5 = 0; 24 goto LAB_0065a618; 25 } 26 FUN_007bb428(); 27 iVar7 = *(longlong *) (extraout_x9 + 0x68); 28 memset(auStack312,0,0xe8); 29 FUN_007be464(auStack312); 30 iVar1 = FUN_0063af64(); 31 if ((iVar1 == 0) (iVar1 = FUN_0061a464(auStack88,0,*unaff_x21), iVar1 == 0)) { 32LAB_0065a600: 33 FUN_007be438(); 34 FUN_0061542c(); 35 uVar5 = 0; 36 } 37 else { 38 FUN_007be458(auStack312); 39 iVar1 = FUN_0063c0e8(); 40 if (iVar1 == 0) goto LAB_0065a600; 41 uVar6 = *(ulonglong *) (local_118 + 0x10); 42 } </pre>	x64
---	-----	---	-----

Decompiled method in x86 vs x64

My normal approach would be to copy the first bytes of FUN_0065a4ec and search for them in-memory while the application is running, [as I did in the previous blogpost](#), so I don't need to find the offset each time. Unfortunately, Frida's Memory.scan seems to [crash](#) on my test app, so for now we'll have to use the offset. (Edit: An alternative approach was posted in the comments of this post, using Process.enumerateRangesSync)

Ghidra uses 0x100000 as the base address of the module, so we have to subtract that from the Ghidra offset, resulting in an offset of **0x55a4ec**.

Opening Ghidra every time works, but it's not that convenient. We can also use binwalk to find the correct offset based on those first bytes of the function:

```

1 # The first bytes of the FUN_0065a4ec function
2 ff 03 05 d1 fc 6b 0f a9 f9 63 10 a9 f7 5b 11 a9 f5 53 12
3 # Find it using binwalk
4 binwalk -R "\xff\x03\x05\xd1\xfc\x6b\x0f\xa9\xf9\x63\x10
5 DECIMAL          HEXADECIMAL      DESCRIPTION
6 -----
7 5612780          0x55A4EC           Raw signature (\xff\x03\x0

```

Let's throw this in a Frida script and test it!

```

1 function hook_ssl_verify_result(address)
2 {
3     Interceptor.attach(address, {

```

```

4      onEnter: function(args) {
5          console.log("Disabling SSL validation")
6      },
7      onLeave: function(retval)
8      {
9          console.log("Retval: " + retval)
10         retval.replace(0x1);
11     }
12 }
13 });
14 }
15 function disablePinning(){
16     // Change the offset on the line below with the bin
17     // If you are on 32 bit, add 1 to the offset to ind
18     // Otherwise, you will get 'Error: unable to inter
19     var address = Module.findBaseAddress('libflutter.so
20     hook_ssl_verify_result(address);
21 }
22 setTimeout(disablePinning, 1000)

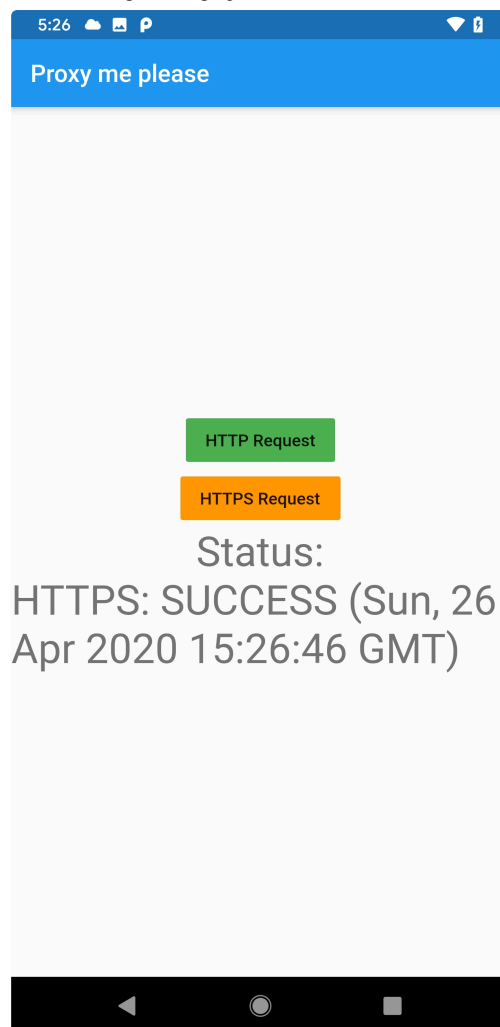
```

Running this file using Frida gives the expected outcome:

```

1  (secenv) → flutter frida -Uf be.nviso.flutter_app -l h
2
3      /_/_/_/_|   Frida 12.8.20 - A world-class dynamic inst
4      | ( _ | |
5      > _ |   Commands:
6      /_/_/_/_|   help      -> Displays the help system
7      . . . .   object?    -> Display information about
8      . . . .   exit/quit  -> Exit
9      . . . .
10     . . . .   More info at https://www.frida.re/docs/hom
11     Spawned `be.nviso.flutter_app`. Resuming main thread!
12     [SM-G950F::be.nviso.flutter_app]-> disablePinning()
13     [SM-G950F::be.nviso.flutter_app]-> Disabling SSL valida
14     Retval: 0x0
15     [SM-G950F::be.nviso.flutter_app]->

```



SSL Verification successfully disabled

Tangent: Why can this app perform cleartext HTTP calls?

My flutter app is making HTTP connections. This is [forbidden by default since Android P](#), and you have to add a Network Security Policy that explicitly allows cleartext request if you still want to do so on Android 9+. My test app does not have a Network Security Policy, so what's going on?

The reason for this is the same reason why these blog posts are necessary: Flutter doesn't use default Android libraries. Because Flutter creates low level sockets and implements the HTTP stack on top of that, the requests never pass by the Android security controls that should prevent cleartext traffic from being used. This is an important thing to keep in mind when auditing the security of Flutter apps, as you might miss things if you're not careful.

TL;DR (ARMv7 and ARMv8)

1. Redirect with ProxyDroid on rooted device since Flutter apps are still proxy-unaware

2. Find the offset using binwalk
3. Use the Frida script to hook the method at that offset

Since the last blogpost, the signature for 32bit also changed, so I've included both signatures.

```

1  # Method signatures for ARMv7 (32bit)
2  2d e9 f0 4f a3 b0 81 46 50 20 10 70
3  2d e9 f0 4f a3 b0 82 46 50 20 10 70
4  # Get the offset
5  binwalk -R "\x2d\xe9\xf0\x4f\xa3\xb0\x81\x46\x50\x20\x1
6  DECIMAL          HEXADECIMAL      DESCRIPTION
7  -----
8  3831160          0x3A7578          Raw signature (\x2d\xe9\x
9  # Method signature for ARMv8 (64bit)
10 ff 03 05 d1 fc 6b 0f a9 f9 63 10 a9 f7 5b 11 a9 f5 53 1
11 # Get the offset
12 binwalk -R "\xff\x03\x05\xd1\xfc\x6b\x0f\xa9\xf9\x63\x1
13 DECIMAL          HEXADECIMAL      DESCRIPTION
14 -----
15 5612780          0x55A4EC          Raw signature (\xff\x03\x

```

Frida script to use the offset:

```

1  function hook_ssl_verify_result(address)
2  {
3      Interceptor.attach(address, {
4          onEnter: function(args) {
5              console.log("Disabling SSL validation")
6          },
7          onLeave: function(retval)
8          {
9              console.log("Retval: " + retval)
10             retval.replace(0x1);
11         }
12     });
13 }
14
15 function disablePinning(){
16     // Change the offset on the line below with the bin
17     // If you are on 32 bit, add 1 to the offset to ind
18     // Otherwise, you will get 'Error: unable to inter
19     var address = Module.findBaseAddress('libflutter.so
20     hook_ssl_verify_result(address);
21 }
22 setTimeout(disablePinning, 1000)

```

And launch it using Frida:


```
1 | frida -Uf hook.js -f be.nviso.flutter_app --no-pause
```

If it still doesn't work, you'll have to figure out the correct method to hook yourself. You can try following [the steps for ARMv7 as described on this blog](#).

About the author

Jeroen Beckers is a mobile security expert working in the Nviso Cyber Resilience team and co-author of the OWASP Mobile Security Testing Guide (MSTG). He also loves to program, both on high and low level stuff, and deep diving into the Android internals doesn't scare him. You can find Jeroen on [LinkedIn](#).

Tagged: Mobile, android, flutter

Published by Jeroen Beckers

[View all posts by Jeroen Beckers](#)

< Three tips for a better IT Acceptable Use Policy

A checklist to populate your Acceptable Use Policy >

21 thoughts on “Intercepting Flutter traffic on Android (ARMv8)”

Pingback: [Intercepting traffic from Android Flutter applications – Nviso Labs](#)

AaYohan

June 6, 2020 at 11:16 pm

Hi Jeroen,

First of all, thanks you for sharing.

I have workaround for sigscan on arm64, rather than using module i'm using memory range as base address and size.

frida -version

12.9.4

```
Process.enumerateRangesSync('r-x').filter(function (m) {  
  if (m.file)  
    return m.file.path.indexOf('libflutter.so') > -1;  
  return false;  
}).forEach(function (r) {  
  Memory.scanSync(r.base, r.size, pattern).forEach(function (match) {  
    hook_ssl_verify_result(match.address);  
  });  
});
```

Loading...

[↩ Reply](#)

Jeroen Beckers

June 8, 2020 at 8:47 am

Thanks for the alternative suggestion! That's a good workaround until the bug is fixed at Frida's side.

Loading...

[↩ Reply](#)

Pingback: [Intercepting Flutter traffic on iOS – Nviso Labs](#)

Andrew Romanov

August 31, 2020 at 4:46 pm

The method doesn't work, even with your test app

Loading...

↩ Reply

Jeroen Beckers

August 31, 2020 at 7:40 pm

I've just retested both 32bit and 64bit ARM, so if the test app doesn't work, it must be something with your device.

Loading...

↩ Reply

Andrew Romanov

September 1, 2020 at 9:38 am

What device do you have?

Loading...

BRUHItsABunny

January 5, 2021 at 9:39 pm

Disclaimer: I'm a noob when it comes to reversing native binaries

I was targeting a 64 bit .so and ended up searching for the string: "ssl_server" instead to find the right function

Inspiration:

https://github.com/google/boringssl/blob/78f15a6aa9f11ab7cff736f920c4858cc38264fb/ssl/ssl_x509.cc#L386

The string was found to be defined only once so it was easy to xref it, which gave me 3 references (twice in the target function and once in the rodata where it was defined).

Also ended up using AaYohan's way of hooking since `Module.findBaseAddress('libflutter.so')` kept returning null for me.

Not sure why it worked, just happy it worked.

Loading...

↪ Reply

Pingback: [Intercept Android SSL Pinning on Flutter Based Android Application – Qlkwej](#)

Alex

June 6, 2021 at 3:27 pm

Thankyou for this post!

I applied a similar procedure for x86_64 architecture which has different method signatures and it worked perfectly

Loading...

↪ Reply

enovella

June 30, 2021 at 10:16 am

Searching the offset with Radare2 is a piece of cake:

```
$ r2 ~/Downloads/flutterCertPinning.apk.unpack/lib/arm64-v8a/libflutter.so
Warning: run r2 with -e bin.cache=true to fix relocations in disassembly
— Of course r2 runs FreeBSD
[0x00270000]> /x ff 03 05 d1 fc 6b 0f a9 f9 63 10 a9 f7 5b 11 a9 f5 53 12 a9 f3
7b 13 a9 08 0a 80 52
Searching 28 bytes in [0x816d00-0x828710]
hits: 0
Searching 28 bytes in [0x6f0000-0x816d00]
hits: 0
Searching 28 bytes in [0x270000-0x6e1ce0]
hits: 1
Searching 28 bytes in [0x0-0x26cc9c]
hits: 0
0x0055a4ec hit0_0
ff0305d1fc6b0fa9f96310a9f75b11a9f55312a9f37b13a9080a8052
[0x00270000]>
```

Loading...

↩ Reply

Pingback: [Null address in Sslpinning bypass of flutter app by using frida – Ask Android Questions](#)

Edward Guan

October 6, 2021 at 4:56 pm

Hello Jeroen,

I have a question about `binwalk` and `Ghidra`. I got two different address by these tools:

<https://imgur.com/a/OUsceBu>

I have changed the base address to 0x0 for Ghidra. Do you know why the binwalk command got the wrong address? Thanks very much.

Loading...

↩ Reply

Pingback: [Proxying Android app traffic – Common issues / checklist \(2023\) – NVISO Labs](#)

luke

April 13, 2023 at 12:39 pm

Thanks for the post.

I didn't understand this part:

"we can see that the OPENSSL_PUT_ERROR macro is called at line 390.

Searching for the number 390 (or 0x186) gives us some results (Search > For Scalars...):"

Why would we deduce that the line number on a c++ program would be present in the binary of the application?

Loading...

↩ Reply

luke

April 13, 2023 at 12:42 pm

Nevermind, I understand now that the OPENSSL_PUT_ERROR macro will need the line number where the error occurred.

Loading...

↩ Reply

Carnival

May 10, 2023 at 11:14 am

Using the APK you provided on Github, but using bingwalk cannot find a corresponding bytecode or any offset

Loading...

↩ Reply

Jeroen Beckers

May 10, 2023 at 11:19 am

Please try the latest version of the script on <https://github.com/NVISOsecurity/disable-flutter-tls-verification> . If it still doesn't work, please open a github issues.

Loading...

↩ Reply

Carnival

May 10, 2023 at 11:41 am

sorry read others blog done

Loading...

↩ Reply

Dastan

January 28, 2024 at 10:50 am

TypeError: невозможно прочитать свойство 'add' со значением null
при disablePinning

Возникает эта ошибка при запуске этого скрипта.

Дальнейшее исследование модулей, загружаемых с использованием
приведенного ниже кода.

```
Process.enumerateModules({  
  onMatch: function(module){  
    console.log('Имя модуля: ' + module.name + " – Базовый адрес: " +  
      module.base.toString());  
  },  
  onComplete: function(){}  
});
```

Было замечено, что libflutter.so там отсутствует.

Загрузка...

↪ Ответить

Йерун Бекерс

29 января 2024 г. в 9:51 утра

Привет, Дастан! Пожалуйста, откройте тикет на github
<https://github.com/NVISOsecurity/disable-flutter-tls-verification> и
приложите APK, который не работает.

Загрузка...

↪ Ответить

Оставить ответ



[Вакансии на домашней странице NVISO](#)

Информация и поддержка

info@nviso.eu

Вас взломали?

csirt@nviso.eu